

Assignment Title:
C Programming Basics (Lab)

Coursework Type:
Individual

Module Name:
ST5003CMD – Operating Systems, Security, And Networks

Submitted By:
Name: Pranisha Bhatta
College ID: 250150

Module Leader:
Name: Rupak Rajbanshi

Table of Content

Introduction **3**
Creating the source file: 3
Stage 1 : Preprocessing **4**
Stage 2 : Compilation **4**
Stage 3 : Assembly **5**
Stage 4 : Linking **5**
Conclusion **6**

C Compilation Process

Introduction

A C program doesn't go directly from source code to executable, GCC actually does this in four separate stages. To understand this better, I created a small program mycode.c with a function hello() that returns a value, and ran it through each stage manually to see what changes at every step.

Creating the source file:

```
pranisha@ubuntu:~/Desktop$ touch mycode.c
```

This creates an empty file called mycode.c.

```
pranisha@ubuntu:~/Desktop$ cat mycode.c
```

Since the file was just created, cat shows nothing confirming it's empty before writing any code into it.

```
pranisha@ubuntu:~/Desktop$ nano mycode.c
```

This opens the file in the nano editor to write the actual C code.

Source code:

```
  GNU nano 8.7.1                                          myfile.c *
#include <stdio.h>

int hello();

int main() {
        int result=hello();
        printf("function return:%d\n", result);
        return 0;
}

int hello() {
        return 25;
}
```

Stage 1 : Preprocessing

```
pranisha@ubuntu:~/Desktop$ gcc -E mycode.c -o mycode.i
```

The preprocessor's job is to handle anything starting with #, so here it expands #include <stdio.h> into the actual header content, producing mycode.i.

```
pranisha@ubuntu:~/Desktop$ ls
mycode.c  myfile.c  myfile.o  newcfile    newcfile.i  newcfile.s  pfile.c  pfile.o  practicefile.c  practicefile.o
mycode.i  myfile.i  myfile.s  newcfile.c  newcfile.o  pfile       pfile.i  pfile.s  practicefile.i  practicefile.s
```

This confirms democomp.i has been created in the directory.

Stage 2 : Compilation

```
pranisha@ubuntu:~/Desktop$ gcc -S mycode.i -o mycode.s
```

Next, the preprocessed file gets compiled into assembly instructions, producing mycode.s. It's still a plain text file, but the content is now low-level assembly rather than C.

```
pranisha@ubuntu:~/Desktop$ ls
mycode.c  myfile.c  myfile.s    newcfile.i  pfile    pfile.o        practicefile.i
mycode.i  myfile.i  newcfile    newcfile.o  pfile.c  pfile.s        practicefile.o
mycode.s  myfile.o  newcfile.c  newcfile.s  pfile.i  practicefile.c  practicefile.s
```

This confirms mycode.s now exists alongside the earlier files.

Stage 3 : Assembly

```
pranisha@ubuntu:~/Desktop$ gcc -c mycode.s -o mycode.o
```

The assembler takes the .s file and converts it into machine code, producing mycode.o. This is no longer readable text, it's a binary object file.

```
pranisha@ubuntu:~/Desktop$ ls
mycode.c  mycode.s  myfile.o  newcfile.c  newcfile.s  pfile.i  practicefile.c  practicefile.s
mycode.i  myfile.c  myfile.s  newcfile.i  pfile       pfile.o  practicefile.i
mycode.o  myfile.i  newcfile  newcfile.o  pfile.c     pfile.s  practicefile.o
```

This confirms mycode.o has been generated.

Stage 4 : Linking

```
pranisha@ubuntu:~/Desktop$ gcc mycode.o -o mycode
```

Finally, the linker takes mycode.o and resolves the reference to printf, pulling in the required library code to produce a complete executable called mycode.

```
pranisha@ubuntu:~/Desktop$ ls
mycode    mycode.o  myfile.i  newcfile    newcfile.o  pfile.c  pfile.s         practicefile.o
mycode.c  mycode.s  myfile.o  newcfile.c  newcfile.s  pfile.i  practicefile.c  practicefile.s
mycode.i  myfile.c  myfile.s  newcfile.i  pfile       pfile.o  practicefile.i
```

This confirms the final executable mycode now appears in the directory.

```
pranisha@ubuntu:~/Desktop$ ./mycode
function return:25
```

Running the program confirms everything worked correctly.

Conclusion

Going through this stage by stage made it clearer that a single gcc file.c -o file command is really doing four different jobs behind the scenes, preprocessing, compiling, assembling, and linking each producing its own intermediate file along the way.